



Convenciones de Codigo y Base de Datos

Idioma, casing, estructura de carpetas y reglas de base de datos para todo el IDEC

Este documento incluye dos partes:

Parte 1 — Convenciones de nombres: idioma (incluye endpoints), casing por tipo de elemento, orden de carpetas y nomenclatura de ramas de Git.

Parte 2 — Convenciones de base de datos: nombres, tipos, claves, indices, auditoria, soft delete y datos espaciales.

Ambas complementan al documento de arquitectura del repositorio y al de decisiones de diseno.

PARTE 1 DE 2



Convenciones de Nombres

Como se nombran las cosas: idioma (incluye endpoints), casing, orden de carpetas y nomenclatura de ramas de Git. Complementa al documento de arquitectura del repositorio y al de decisiones de diseño.

REGLA GENERAL

A partir de ahora todo el código **nuevo** se escribe en inglés: variables, funciones, clases, archivos, carpetas, comentarios, docstrings y **endpoints**. Reemplaza a la regla anterior ("español para negocios, inglés para técnico"). El código existente (casi todo el repo hoy) queda en español como deuda histórica hasta el refactor grande — un cambio de código aparte y coordinado, no algo para ir tocando línea por línea sin querer. El refactor grande incluye el esquema de la base de datos (tablas/columnas) cuando se coordina con el equipo de base de datos.

Actualización de regla — endpoints en inglés. Los endpoints también se escriben en inglés, sin excepción. Mantenerlos en español mientras el resto del sistema (dominio, backend, base de datos) pasa a inglés rompe la consistencia de un esquema de nombres ya planeado: la ruta dejaría de reflejar el dominio y el recurso reales del código que la implementa. Los endpoints **existentes** en español siguen como deuda histórica (igual que el resto del código legado) hasta el refactor grande; los endpoints **nuevos** se crean directamente en inglés.

1 Idioma

Regla: inglés para todo el código — nombres de dominio, módulo, entidad, caso de uso, función, variable, clase, archivo, carpeta, comentario, docstring y nombre de tabla/columna. Sin excepción por ser "de negocio": ya no se traduce *Campania* a *Campaign* solo en la capa técnica, se traduce en todos lados — incluida la base de datos, cuando se llega a esa parte del refactor.

Lo que se mantiene en español (excepciones permanentes, no deuda a corregir)

Se mantiene en español	Por que
Texto visible en la UI: etiquetas, botones, mensajes de error, marcadores de posición	Los usuarios reales son funcionarios municipales de Cochabamba que hablan español — traducir la interfaz sería un cambio de producto, no de convención de código
Valores de datos ya cargados que son contenido, sin esquema	Es dato, no un identificador que nombremos nosotros — se migra aparte si alguna vez hace falta, no es parte de este refactor de código

Ejemplo del nuevo estilo (código nuevo, a partir de hoy)

```
class CampaignNotFoundException(DomainException): ...
def list_campaigns(...): ...
user_id, role_id, area_id
# Comment explaining a non-obvious decision goes in English too.

function fetchWmsLayers() { ... }
function useSecurityData() { ... }
import { Button, Modal } from '@shared/ui'
// Same here: comments in English.
```

Los textos que si quedan en español (ejemplo real, sin cambios): `toast.warn('Dibuja un area (poligono o rectangulo) en el mapa.')` . `<Button>Guardar</Button>`

2 Casing por tipo de elemento

El casing no cambia. Ejemplos ya en el estilo nuevo, en inglés — incluidos endpoints y base de datos, que son el objetivo del refactor, no el estado de hoy.

Backend (Python)

Elemento	Convencion	Ejemplo (objetivo)
Archivo / modulo	snake_case.py	sql_role_repository.py
Funcion / metodo / variable	snake_case	list_wms_layers, user_id
Clase	PascalCase	InternalRoleModel, DetectChangesUseCase
Constante	UPPER_SNAKE_CASE	SECURITY_SYSTEM, DETECTION_ENGINE
Tabla/columna de base de datos	snake_case	internal_role, keycloak_sub
Punto final (ruta HTTP)	/api/<domain>/<resource> en plural, snake_case/kebab-case	/api/constructiondetection/campaigns
Código de permiso	resource.action	"campaigns.read", "security.edit"

Interfaz de usuario (JS/JSX)

Elemento	Convencion	Ejemplo
----------	------------	---------

Archivo de componente	PascalCase.jsx, un componente por archivo	RoleFormModal.jsx
Archivo de utilidad/servicio/tienda	camelCase.js	securityStore.js, constructionDetection.api.js
Componente React	PascalCase	ReportTable, UserAssignmentModal
Gancho (hook)	useSomething (camelCase con prefijo use)	useSecurityData, useDetectionMap
Funcion / variable	camelCase	fetchWmsLayers, roleBeingEdited
Constante de modulo	UPPER_SNAKE_CASE	INITIAL_PARAMS, SECURITY_SYSTEM_ID
Carpeta de dominio	identico en nombre al dominio del backend	domains/constructiondetection/ en ambos lados

3 Orden de carpetas

Mismo esqueleto que ya define el documento de arquitectura — aqui solo como referencia rapida de una sola pantalla.

⚠ Los nombres de carpeta de dominio actuales (`domains/seguridad/` , `domains/deteccionconstrucciones/` , `domains/geoextraccion/`) siguen como estan hoy hasta el refactor — no renombrar una sola carpeta de dominio suelta, ese cambio se hace completo o no se hace, porque toca rutas de API, la base real y el nav del frontend a la vez.

Backend — dentro de cada dominio (backend/app/domains/<domain>)

```
domain/           # entities + ports (interfaces) – sin FastAPI ni SQLAlchemy
  entities/
  ports/
  services/       # logica de negocio pura, sin dependencias pesadas (opcional)
application/
  use_cases/      # un caso de uso = una accion de negocio
infrastructure/
  models.py       # modelos ORM
  sql_*_repository.py
  /              # ej. wms/, predios/, detector/, imaging/, jobs/
presentation/
  endpoints/
  schemas/
  deps.py
  router.py
contracts/        # lo unico que otros dominios pueden importar de este
tests/
```

Frontend — dentro de cada dominio (Frontend/src/domains/<domain>)

```
pages/           # una pantalla = un archivo
components/      # compartido SOLO dentro de este dominio
hooks/
api/             # cliente HTTP del dominio
utils/
data/            # store/estado del dominio (si aplica)
routes.jsx       # se registra en el router raiz
```

💡 **Orden de importaciones** dentro de un archivo (arriba hacia abajo): librerias externas → @/core / @/shared → dominio propio (./ relativo) → tipos/constantes locales.

4 Nomenclatura de ramas (Git)

Misma regla de idioma que el resto del codigo: los nombres de rama se escriben en **ingles**, sin excepcion. Las ramas ya existentes con nombres en espanol no se renombran retroactivamente; el nombre en ingles aplica desde ahora para toda rama nueva.

Formato

```
<type>/<short-description-in-kebab-case>
```

Opcionalmente, cuando exista numero de ticket o issue, se antepone justo despues del tipo:

```
<type>/<ticket-id>-<short-description-in-kebab-case>
```

✓ CORRECTO

✗ INCORRECTO

- feature/campaign-property-filter
- feature/CAT-142-wms-layer-toggle
- fix/auth-token-expiry
- hotfix/production-login-crash
- release/1.4.0
- chore/update-dependencies

- Feature/CampaignFilter
- nueva-funcionalidad-campanas
- fix_bug_login
- Diego/pruebas
- arreglo-urgente
- RAMA NUEVA

Tipos de rama

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad o mejora de negocio	feature/campaign-property-filter
fix/	Correccion de un bug que no es critico en produccion	fix/wms-layer-render
hotfix/	Correccion urgente directamente sobre produccion	hotfix/auth-token-expiry
release/	Preparacion de una version para despliegue	release/1.4.0
refactor/	Cambio de estructura interna sin alterar comportamiento	refactor/campaign-repository
chore/	Mantenimiento: dependencias, configuracion, tooling	chore/update-dependencies
docs/	Cambios solo de documentacion	docs/database-conventions
test/	Cambios enfocados solo en pruebas	test/campaign-service-coverage

⚠ No mezclar tipos ni inventar un prefijo nuevo para casos que ya encajan en la tabla — la misma regla de consistencia que aplica a nombres de tablas y columnas (ver Parte 2, seccion 24) aplica aqui. Si un tipo no existe todavia y se necesita, la decision se documenta antes de usarlo por primera vez.

Ramas principales (no llevan prefijo)

Rama	Rol
master	Codigo en produccion. Solo se actualiza via release/ o hotfix/
develop	Integracion de features antes de un release

PARTE 2 DE 2



Convenciones de Base de Datos

Reglas de nombres, tipos y estructura para todo esquema nuevo o modificado durante el refactor.

REGLA GENERAL

El código nuevo se escribe en inglés y la base de datos nueva también utiliza nombres en inglés. El esquema existente que actualmente está en español queda como deuda histórica hasta el refactor grande y coordinado con el equipo de base de datos. Los datos existentes cuyo contenido sea español **no** se traducen por esta convención — esta regla aplica a los nombres que nosotros definimos para el esquema, no al contenido almacenado.

1 Idioma

Todo nombre **nuevo** definido en la base de datos se escribe en inglés: tablas, columnas, índices, constraints, claves primarias, claves foráneas, vistas, funciones, procedimientos, sequences y schemas (cuando corresponda).

✓ CORRECTO

- user
- role
- permission
- campaign
- property
- construction
- created_at
- campaign_id

✗ INCORRECTO

- usuario
- rol
- permiso
- campania
- predio
- construccion
- fecha_creacion

Datos almacenados

El contenido real almacenado en la base de datos **no** forma parte de esta convención. Los valores son datos de negocio y pueden permanecer en español porque están destinados a usuarios de la aplicación.

```
table: campaign
  id: 15
  name: "Deteccion de construcciones 2026"
  description: "Campana realizada en el Distrito 7"
```

2 Casing y nombres

Tablas

Convención: **snake_case** — se recomienda utilizar nombres en **singular** para representar entidades. No utilizar prefijos heredados como **tbl_**, salvo necesidad técnica explícita y documentada.

✓ CORRECTO

- user
- role
- permission
- campaign
- property
- construction
- inspection

✗ INCORRECTO

- Users
- Roles
- campaigns
- Predios
- tbl_users
- tbl_campaign

Columnas

Convención: **snake_case**

✓ CORRECTO

- id
- name
- status
- created_at
- is_active
- user_id
- campaign_id

✗ INCORRECTO

- UserId
- userId
- USER_ID
- idUsuario
- fechaCreacion

3 Primary Keys

Por defecto, cada tabla debe tener una clave primaria. El nombre estandar de la columna sera `id`

```
user
  id
  username
  created_at

campaign
  id
  name
  created_at
```

💡 La estrategia de tipo de identificador debe ser **consistente** en todo el sistema. Decision inicial recomendada: `BIGINT` para identificadores internos, salvo razon para usar UUID. Cuando una entidad requiera UUID (integracion, distribucion, exposicion externa), la decision debe documentarse. No mezclar estrategias arbitrariamente dentro del mismo dominio.

4 Foreign Keys

Las claves foraneas utilizan el nombre de la entidad seguido de `_id`

```
campaign
  id
  name

property
  id
  cadastral_code

campaign_property
  campaign_id
  property_id
```

✓ CORRECTO

- `user_id`
- `role_id`
- `campaign_id`
- `property_id`

✗ INCORRECTO

- `id_campaign`
- `campaignId`
- `idCampaign`
- `fk_campaign`

5 Tablas de relacion

Las relaciones muchos-a-muchos se representan mediante una tabla intermedia. El nombre se construye con las entidades relacionadas:

Entidad A	Entidad B	Tabla de relacion
user	role	user_role
role	permission	role_permission
campaign	property	campaign_property

⚠ Cuando el orden pueda ser ambiguo, se debe mantener un orden **consistente** definido por el dominio: `user_role` siempre — no alternar entre `role_user` y `user_role` para el mismo concepto.

6 Fechas y horas

✓ CORRECTO

- `created_at`
- `updated_at`
- `deleted_at`
- `started_at`

✗ INCORRECTO

- `date`
- `fecha`
- `creation_date`
- `created`

- `completed_at`
- `published_at`

- `last_update`

Cuando se almacene **solo fecha**: `birth_date`, `inspection_date`, `effective_date`. Cuando se necesite fecha y hora: `created_at`, `updated_at`, `started_at`.

⌚ Las fechas y horas deben almacenarse utilizando una estrategia consistente de zona horaria definida por el proyecto.

7 Valores booleanos

Utilizar nombres que expresen claramente una condicion:

✓ CORRECTO

- `is_active`
- `is_deleted`
- `is_verified`
- `is_enabled`
- `has_geometry`
- `has_document`

✗ AMBIGUO — EVITAR

- `active`
- `status`
- `flag`
- `value`
- `enabled_value`

```
user
  id
  username
  is_active
```

8 Estados

Cuando una entidad tenga diferentes estados de negocio, utilizar una columna explicita: `status`

```
campaign
  id
  name
  status      → draft | running | completed | cancelled
```

✗ No utilizar multiples columnas booleanas para representar estados mutuamente excluyentes (`is_draft`, `is_running`, `is_completed`, `is_cancelled`), salvo razon de negocio especifica.

9 Texto

Tipo	Cuando usarlo	Ejemplo
VARCHAR	Longitud maxima conocida y relevante	<code>username VARCHAR(100)</code> <code>code VARCHAR(50)</code>
TEXT	Contenido sin longitud maxima razonable	<code>description TEXT</code> <code>error_message TEXT</code>

⚠ No utilizar `TEXT` indiscriminadamente para todos los campos.

10 Valores numericos

Tipo	Uso
INTEGER / BIGINT	Cantidades enteras
NUMERIC	Precision decimal – amount, price, tax, area, percentage, coordinates

✗ No utilizar tipos de punto flotante para valores monetarios cuando se requiera precision exacta.

11 Auditoria

Las tablas de negocio deben incluir, cuando corresponda:

Columna	Cuando
created_at / updated_at	Siempre que aplique trazabilidad de cambios
created_by / updated_by	Cuando se requiere saber quien realizo la operacion
deleted_at / deleted_by	Cuando se utiliza eliminacion logica

```
campaign
  id
  name
  status
  created_at
  created_by
  updated_at
  updated_by
```

💡 No todas las tablas requieren obligatoriamente todas las columnas de auditoria. La necesidad se define segun la naturaleza de la entidad.

12 Soft Delete

Cuando una entidad requiera eliminacion logica, se utilizara `deleted_at` (opcionalmente `deleted_by`).

Valor	Significado
NULL	Registro activo
2026-09-08 14:30:00	Registro eliminado logicamente

✗ No utilizar `is_deleted` como mecanismo principal cuando la fecha de eliminacion sea relevante. El uso de soft delete debe ser una decision del dominio, no una regla automatica para todas las tablas.

13 Unique Constraints

Deben representar reglas reales del negocio o necesidades tecnicas claras: `UNIQUE(username)` , `UNIQUE(cadastral_code)` .

✗ No utilizar `UNIQUE` simplemente para evitar duplicados sin conocer la regla de negocio.

14 Indices

Se crean en funcion de las consultas y relaciones reales. Las foreign keys usadas frecuentemente para busquedas o joins deben evaluarse para indexacion.

Tipo	Patron	Ejemplo
Indice normal	<code>ix_<table>_<column></code>	<code>ix_campaign_status</code>
Indice unico	<code>ux_<table>_<column></code>	<code>ux_property_cadastral_code</code>

15 Primary Key y Foreign Key Constraints

Las constraints deben tener nombres explicitos y consistentes:

Constraint	Patron	Ejemplo
------------	--------	---------

Primary key	pk_<table>	pk_campaign
Foreign key	fk_<table>_<referenced_table>	fk_campaign_property_campaign
Unique	uq_<table>_<column>	uq_user_username
Check	ck_<table>_<rule>	ck_campaign_status

16 ? Nullability

Las columnas deben permitir `NULL` solamente cuando la ausencia del valor sea válida para el negocio — nunca como sustituto de valores desconocidos cuando el dominio requiera una condición explícita.

```
created_at NOT NULL
deleted_at NULL    ← correcto si se usa eliminación lógica
```

💡 La decisión de `NULL` / `NOT NULL` debe formar parte del diseño de la entidad.

17 📖 Catálogos y valores controlados

Para valores que forman parte de un catálogo de negocio, decidir explícitamente entre: tabla de catálogo, enum, constraint o valores controlados por aplicación.

✗ No crear tablas de catálogo automáticamente para cualquier conjunto pequeño de valores.

Cuando el catálogo pueda cambiar por configuración o administración municipal, se recomienda una tabla — por ejemplo: `construction_type`, `property_type`, `inspection_status`.

18 🌐 Base de datos espacial

Para entidades espaciales se debe definir explícitamente:

- Tipo de geometría
- Dimensionalidad
- SRID
- Sistema de referencia
- Índice espacial
- Reglas de validación geométrica

```
property
  id
  cadastral_code
  geometry
```

✗ No se deben almacenar geometrías con SRID desconocido o mezclando sistemas de referencia sin una decisión explícita.

19 🏠 Tablas espaciales de Catastro

Para entidades como `property`, `construction`, `block`, `district`, `zone`, `road`, la geometría debe considerarse parte del modelo de datos y no simplemente un atributo adicional. Cuando una entidad tenga geometría, se debe definir:

geometry_type	srid	geometry_column	spatial_index
Ejemplo conceptual – <code>property(id, cadastral_code, area, geometry, created_at, updated_at)</code>			

💡 La validación de geometrías debe realizarse antes de persistir información cuando la integridad espacial sea un requisito del dominio.

20 Versionado e historial

Las entidades catastrales pueden requerir historial debido a cambios en el tiempo (`property` , `property_owner` , `property_geometry` , `construction` , `land_use` , `assessment`). No se debe sobrescribir informacion historica cuando el negocio requiera conocer su evolucion.

La estrategia concreta puede utilizar: `history tables` , `version columns` , `effective_from` / `effective_to` , o `event records` , segun la decision de disenio correspondiente.

⚠ Esta decision debe documentarse **antes** de implementar un modelo historico.

21 Migraciones

Los cambios estructurales de la base de datos deben realizarse mediante migraciones versionadas. Cada cambio debe poder identificarse y reproducirse.

migration → development → testing → production

✗ No se debe modificar manualmente el esquema de produccion como mecanismo normal de despliegue.

Cuando un cambio implique perdida de informacion, debe existir una estrategia explicita para: respaldo, migracion de datos, validacion, rollback/recuperacion y despliegue.

22 ORM y base de datos

Los modelos ORM **no** definen por si solos el modelo de dominio. La arquitectura mantiene separadas:

Domain Entity → Repository Port → ORM Model → Database

`domain/entities/campaign.py` representa la entidad de dominio, mientras que `infrastructure/models.py` representa su persistencia. El modelo ORM puede contener detalles especificos de SQLAlchemy o del motor de base de datos sin introducir esas dependencias en la capa de dominio.

23 Compatibilidad con codigo existente

El esquema actual contiene nombres historicos en espanol. Estos nombres **no** deben modificarse individualmente como parte del desarrollo normal.

Hoy (permanece temporalmente)	Refactor grande →
predio	property
propietario	owner
fecha_registro	registered_at

El refactor debe realizarse como una iniciativa coordinada:

Database → Backend → API → Frontend → Integrations

💡 Las rutas HTTP **existentes** en espanol permanecen temporalmente como deuda historica, igual que el resto del esquema legado. Las rutas **nuevas** se crean en ingles desde ahora (ver Parte 1, seccion 1).

24 Regla de consistencia

La prioridad es la consistencia. Una vez definida una convencion, no se debe crear una nueva variante para un modulo particular.

Por ejemplo, si se utiliza `created_at` / `updated_at` , no crear posteriormente `creation_date` / `last_modified` para otra entidad sin una razon documentada. La misma regla aplica a: IDs, Foreign Keys, timestamps, boolean fields, indexes, constraints y spatial columns.

25 Ejemplo completo

Una tabla nueva siguiendo estas convenciones:

```
campaign
  id          BIGINT PRIMARY KEY
  name        VARCHAR(200) NOT NULL
  description  TEXT
  status      VARCHAR(50) NOT NULL
  started_at  TIMESTAMP
  completed_at TIMESTAMP
  created_at  TIMESTAMP NOT NULL
  created_by  BIGINT
  updated_at  TIMESTAMP NOT NULL
  updated_by  BIGINT
```

Con indices y constraints: `pk_campaign` · `ix_campaign_status` · `fk_campaign_created_by` · `fk_campaign_updated_by`

```
campaign_property
  campaign_id BIGINT NOT NULL
  property_id BIGINT NOT NULL
  created_at  TIMESTAMP NOT NULL
```

Con: `pk_campaign_property` · `fk_campaign_property_campaign` · `fk_campaign_property_property` · `ix_campaign_property_campaign_id` · `ix_campaign_property_property_id`

26 Regla final — Checklist antes de crear una tabla

- ✓ Nombre en ingles
- ✓ snake_case
- ✓ Primary key definida
- ✓ Foreign keys definidas
- ✓ NULL / NOT NULL definido
- ✓ Tipos de datos adecuados
- ✓ Auditoria definida cuando corresponda
- ✓ Indices evaluados
- ✓ Constraints definidas
- ✓ Relaciones definidas
- ✓ Reglas de eliminacion definidas
- ✓ Reglas de unicidad definidas
- ✓ Requisitos espaciales definidos, si corresponde
- ✓ Migracion preparada
- ✓ Impacto sobre backend y frontend revisado

La base de datos debe evolucionar mediante cambios coordinados y versionados, evitando decisiones locales que creen convenciones diferentes dentro del IDEC.